

1 What the extension's backend actually is (read from v2.1.220, 260731)

the extension is a HOST, not an agent

```
+-----+ bundles +-----+ spawns +-----+
| webview UI | -----> | Agent SDK | -----> | claude CLI |
+-----+               +-----+               +-----+
no new agent      no new protocol    only a shell around an engine we run
```

The extension does not implement an agent, and it does not implement a protocol either. It BUNDLES the TypeScript Agent SDK and drives it, which `extension.js` gives away verbatim in its option names: `pathToClaudeCodeExecutable`, `canUseTool`, `includePartialMessages`, `settingSources`, `permissionPromptToolName`, `forkSession`, `sessionMirror`. Those are the same option names our Python `claude_agent_sdk` exposes, because it is the same SDK in two languages.

<code>webview/index.js</code>	5MB	the chat UI: bubbles, diffs, tool cards. web chat, never a terminal
<code>extension.js</code>	2.5MB	the host: bundled Agent SDK + VS Code glue + the IDE bridge server
<code>resources/native-binary/claude</code>	245MB	a bundled copy of the ORDINARY CLI, spawned as a subprocess
<code>~/.claude/ide/<pid>.lock</code>		the bridge handshake: the EXTENSION listens on a WebSocker and the CLI dials OUT to it (transport: "ws" + authToken)

That 245MB file is not a special build and not a second product (JL asked, 260731). It is the Claude Code CLI compiled to one Mach-O executable with the Node runtime and every dependency baked in, which is the whole reason for the size. Verified on this machine: `shasum -a256` matches `~/.local/share/claude/versions/2.1.220` byte for byte, which is exactly what `claude` on the `PATH` symlinks to. The extension carries a copy so it still works for someone who never installed the CLI; that is the only thing the copy buys, and we need no copy because `serve.py` already spawns the one on the `PATH`. The spawn line is fixed and short, then one flag per set option: `claude --output-format stream-json --verbose --input-format stream-json`. Everything else is mechanical translation, the same table our options would produce: `--model`, `--effort`, `--thinking adaptive|disabled`, `--max-turns`, `--max-budget-usd`, `--setting-sources=...`, `--allowedTools`, `--disallowedTools`, `--permission-mode`, `--mcp-config`, `--resume=<sid>`, `--session-id=<sid>`, `--fork-session`, `--include-partial-messages`. This page owns the DRAWER and nothing else: the three permission tiers, streaming, rendering, cost, its session handling, and every part of it a reader touches. The rules it obeys about who may hold a session are QD1's, the real terminal is QD3 and its form on a small screen is QD4, and the shell that puts the drawer in its own frame is QD5. Whether a reader can TRUST what the drawer shows, along BINDING · TURN · CONTINUITY · HANDOVER · INTERRUPTION, is tested by QF4, which is also where "were these fixes ever clicked in a browser" belongs. How well this page is WRITTEN is QB4's grammar and QF1's checker, recorded here 260801 only because JL asked it on this page (" follow guideline ... Q ") and the answer is that it is not this page's to own.

2 The protocol on that pipe

ONE pipe, four kinds of newline JSON

<code>assistant + deltas</code>	<code>--></code>	the half the drawer already renders
<code>control_request</code>	<code>--</code>	the CLI ASKING: a permission prompt

```

control_response      -->  our allow/deny, matched by request_id
control_cancel_request -- plus keep_alive . transcript_mirror
setting can_use_tool is what switches the control half ON

```

One pipe carries four kinds of traffic, all newline JSON. Assistant output and partial deltas come up as ordinary messages; that is the half our drawer already renders. The other three are the half we do not use yet: `control_request` (the CLI asking, which is how a permission prompt arrives), `control_response` (our answer, matched by `request_id`), and `control_cancel_request`. `keep_alive` and `transcript_mirror` frames ride the same channel. Setting `canUseTool` is what turns the channel on: the SDK appends `--permission-prompt-tool stdio`, and every gate decision then travels as a control message rather than a side channel.

3 The one difference that matters, named exactly

```

extension  > ONE process, MANY turns      instant follow-up
turn1 --+
turn2 -+-->  one live claude -->
turn3 --+

serve.py, before M1 > one process PER TURN  8.1s first token .  ~$0.9
turn1 -->  boot  ~150 skills -->  -->  dropped
turn2 -->  boot  ~150 skills -->  -->  dropped
the WHOLE gap: we drop the client every POST

```

The extension’s read loop runs ONCE for the life of a session and pushes each new user turn into the live process with `InputStream.enqueue(...)`. That is what `--input-format stream-json` buys: `stdin` is a STREAM of turns, not a single prompt, so one process serves the whole conversation. Our `ClaudeSDKClient` has the identical capability, and its own docstring names our exact use case (“Building chat interfaces or conversational UIs”, “Multi-turn conversations with context”). `serve.py` threw it away: `chat()` opened `async with ClaudeSDKClient(...)` inside a per-POST `anyio.run(run)`, so every message connected, ran one turn, and disconnected. That single line was the whole “not that good”: the 8.1s first token and the near-0.9full – *tier messages were both the cost of reconnecting and reloading the 150–skill registry per message. The blocker was equal lived event–looph thread owning every live client, with queues in and out; the HTTP handler stops owning the loop and be*

4 What “exactly the same as the extension” costs, in milestones

```

M1 the session host  -- unlocks everything below --+
one daemon thread . one loop . SESSIONS[question] |
                                                    +--> M2 streaming verbs
                                                    |   interrupt . set_model
                                                    |   set_permission_mode
                                                    |   get_context_usage
                                                    +--> M3 rewind_files()
                                                    +--> M4 @-mentions . plan mode

M4 is OURS to build; M2 and M3 are option flips once M1 lands

```

- M1 · the session host A daemon thread runs one asyncio loop for the process's life; `SESSIONS[question]` → `{client, inbox, outbox}`. `chat()` stops calling `anyio.run`; it submits the message to the loop, then drains that session's outbox into the existing NDJSON stream, so the browser side changes not at all. Idle reaping and the QD1 HOLD keep their current rules. - M2 · the streaming-mode verbs, free once M1 lands Four `ClaudeSDKClient` methods work ONLY in streaming mode and are unreachable today: `interrupt()` replaces our flag that waits for the next message boundary, `set_model()` and `set_permission_mode()` switch mid-conversation with no reboot, and `get_context_usage()` gives the drawer a real context meter. - M3 · rewind, the feature we had parked `rewind_files(user_message_id)` is checkpoints and rewind, listed on this page as "parked, it fights the LAW". It stops fighting: the amended QD1 Law already allows many sessions per question, and rewind operates inside one session rather than forking a second history. It needs `enable_file_checkpointing=True` plus `replay-user-messages`, both option flips. - M4 · the two UI affordances that are ours to build @-file mentions need a picker over the repo and a path injected into the message. Plan mode is `--permission-mode plan`, which M2's `set_permission_mode()` already reaches.

5 Could we just use 'extension.js', and should we go JS? (JL asked 260731)

"the JS version" is THREE things, with three answers

<code>extension.js</code>	<code>require("vscode")</code> throughout . cannot even LOAD outside the IDE
the npm SDK	the SAME SDK in another language . 0 capability gained
stay Python	47 options already cover every flag the extension emits
the slowness is	the DROPPED CLIENT, not the language

Three different things get called "the JS version", and they have three different answers. - `extension.js` itself: NO, and not for taste reasons. It is a VS Code extension-host module that calls `require("vscode")` throughout, exports no public API, and ships as one 2.5MB minified bundle. Outside VS Code the `vscode` module does not exist, so the file cannot even load. - The SDK it bundles: yes, reusable, and it is on npm as `@anthropic-ai/claude-agent-sdk`. But that is the SAME SDK as our Python one, so adopting it is a language change, not a capability change. - Parity, checked rather than assumed: `ClaudeAgentOptions` in Python carries 47 options covering every flag the extension's arg builder emits, including `thinking`, `effort`, `task_budget`, `can_use_tool`, `include_partial_messages`, `fork_session`, `skills`, `sandbox`, `plugins`, and `enable_file_checkpointing`, plus `extra_args` as the escape hatch for any flag it has not mapped. There is no capability we would gain by switching. So the recommendation is to stay in Python, and the decisive evidence is what else `serve.py` is (JL asked 260731, "what does `serve.py` do besides the claude code?"). Measured 260731 it was 2938 lines across 20 HTTP routes plus the terminal WebSocket, the chat one job of seven; the monolith has since been split, so today `cli/serve.py` is a 496-line HTTP router and the seven jobs live as `live/` modules (`chat.py`, `term.py`, `activity.py`, `write.py`, `xcal.py`, `structure.py`, `shell.py`, `home.py`). Counting lines of method body per area, as measured then:

443	<code>excalidraw</code>	proxy the app . per-page frames . save scenes . hydrate and stash embedded images . attach a drawing to a page (QB4b, QD5)
414	<code>chat</code>	the Claude Code bridge (QD2)
390	<code>activity</code>	a SQLite focus-time database: spans per board, group, page and actor, day-part aggregation, cross-board stats (QD6)
334	<code>write-back</code>	every comment, lane, edit, discussion and resolve landed as a

		typed line under its exact anchor sentence	(QB8)
322	terminal	the PTY serve.py now owns, its WS terminus, ring buffer, resize and reaping	(QD3)
291	http	routing, headers, target resolution, and calling build.py to regenerate board.html after every write	
48	image paste	. HOLD locking . structure edits (add Q, add group, archive)	
~670	module level	the four rules texts, prime_context, tool_brief, _slugify, structure_op, the PTY helpers	

Going JS therefore means porting an Excalidraw round-trip, a SQLite database, a sentence-anchor engine, and a PTY, none of which touch Claude Code, in order to change the language of the one part that already has full parity. `build.py` `check.py` `xcal.py` `lanes.py` `skillpage.py` are Python as well. The decisive point: the slowness is not Python against JS, it is that we drop the client every POST, so a language switch alone would fix nothing while holding the client fixes everything.

6 The goal, stated properly: migrate the plugin ONTO the board (JL 260731)

VS Code plugin		this board	state
webview/index.js	the chat UI	-> board.html's drawer	
extension.js	the host	-> serve.py	
vscode.postMessage	the wire	-> POST /_board/chat + NDJSON	
bundled Agent SDK	the engine	-> claude_agent_sdk (same SDK)	
the claude binary		-> the same binary, from PATH	
the workbench	one long-lived UI	-> QD5's shell: the chat frame is its own document, and it docks	260802
the RETAINED webview	a hidden panel keeps its state	-> the ring: a turn survives its reader, and a returning reader re-attaches at a cursor	A9.1
IDE bridge (WebSocket)		-> nothing yet, and this is the interesting one	

The frame is not "make the drawer more like the plugin". It is that the board becomes the plugin, and the mapping is one to one at every layer. The IDE bridge exists so the session can reach EDITOR surfaces: a diff in a real tab, the current selection, the language server's diagnostics. On this board the editor IS the board page, so the equivalent already half exists and has different names: the sentence address is the selection, QB8's lanes are the annotations, and `check.py`'s output is the diagnostics. That is the one place where migrating means translating rather than copying, and it is where the board can end up better than the plugin rather than merely equal to it.

7 What stays VS Code only, and does not matter

the IDE bridge is the ONE piece we cannot copy
it exists to reach EDITOR surfaces: a diff in a real tab
 the current selection
 the language server's diagnostics
EXACT at the engine + protocol layer
deliberately DIFFERENT at the surface layer the alignment line we hold

The IDE bridge is the one piece we cannot copy, because it exists to put things in EDITOR surfaces: a diff in a real editor tab, the current text selection, the language server's diagnostics. The drawer answers the same need in its own surface and already ships the important half, the diff preview at the permission gate. So "exactly the same" is exact at the engine and protocol layer, and deliberately different at the surface layer, which is the alignment line this page already holds.

8 What a drawn interface has to be given

```
FOUR things nobody gave the drawer          22 defects . 16 2 4
THE VIEWPORT IS BORROWED          it is fixed OVER a page it does not control
THE VIEW IS REGENERATED          build.py bakes it into every page, so it dies with each
THE RECORD IS WRITTEN ONCE       at `done`, the last thing the stream ever sends
THE CONTROLS WERE NEVER GIVEN a terminal has them for free; a drawn surface does not
not four UI complaints: state a reader can see has to be PUT somewhere, and
    three of these four rows are one mechanism away from each other (S9)
```

JL asked twice on 260801 whether the day's problems should become a division named for the surface, something like Mobile Usage or UI Experience, and the first answer here was no, on the grounds that filing them under UI would file architecture under taste. That answer was half right: the grouping by OWNER stands, because every defect arrived as a feeling about the interface and turned out to be a fact about where state lives, but the NAME was a metaphor that told a cold reader nothing. The subject is the interface, and the Opening already names why it needs one: a terminal inherits its behaviour, a drawn interface has to be given all of it, and this section is the list of what nobody gave it. The fourth row is new and is what the rename makes room for: rows one to three are state a reader can lose, row four is an affordance a reader never had, and both are things a drawn surface has to be handed. Scope, corrected on JL's ask 260801 ("focus on the SDK GUI version... the problems for QD2 only"): everything below is the DRAWER's own behaviour. Two classes that were briefly filed here have gone back to their owners, and the Boundary now names both: how well this page is WRITTEN is QB4's grammar and QF1's checker, and whether the drawer's fixes were ever clicked in a browser is QF4, which already exists as "Driving the talk layer: the SDK chat version and the TUI chat version". Every row below is a thing JL hit, read off both session transcripts rather than from memory (ccda0c28-ef7e-47e0-a7e1-c13abc4f4cea + 8c9903ba-dadb-4f00-bdd1-823986cac937, 98 user messages, 260723 14:36 → 260801 20:43).

```
THE VIEWPORT IS BORROWED          4 . 3 1
a wheel over the drawer scrolled the PAGE behind it
opening landed at the oldest message, not the newest
scrolling up during a live turn snapped the reader back down
below 820px it overlays instead of docking; Page shipped; R3 `QD5`'s
    shell, open only for a page opened alone
+ the phone is the usage mode, not the edge case (JL 20:16)
```

```
THE VIEW IS REGENERATED PER PAGE    7 . 5 1 1
close reopen rebuilt every bubble; the flash read as janky
reopening mid-turn did nothing: the guard returned above `.on`
a reload rebuilt the chat box while the terminal held the session
a replayed session does not paint like a live one
+ markdown . tool cards . timestamps landed; gate diffs + thinking owed
```

starting a new session leaves the page unchanged, with no switch
 + closed by Sessions + New session (A8.8, A8.10)
 history has no selectable turn boundary, so two turns cannot be compared
 the drawer is generated into every page at all answered by `QD5`'s shell (S9 R2)

THE RECORD IS WRITTEN AT EXACTLY ONE MOMENT 6 . 6
 the reply appeared only after sending the NEXT message
 the drawer never re-asked the server: syncFromServer had one caller
 a cut-short turn dropped the text that had already arrived
 a group's history was written under `` and read under `G:<id>`
 an in-flight turn dies with the HTTP response that carries it S9 R1, closed
 + navigate away . switch a setting . let it run long
 a ten-minute turn hits the HTTP timeout S9 R1, closed

THE CONTROLS WERE NEVER GIVEN 5 . 2 3
 Sessions, as a composer tab rather than a fold
 the context-usage meter: ctx 38% (62k/200k)
 typing `@` does not pull a repo file into the message
 no plan-mode toggle for a read-only planning turn
 two named sessions on one page cannot be live at once
 + blocked on a `QD1` ruling: HOLD keys on the SCOPE

Read down the column and four remain of the original eight; the rest were closed by §9's R1 and QD5's shell. One is QD1-blocked, one is the turn-boundary compare, and only two are genuinely their own work: @-mentions and plan mode, which are the two the page has always called chrome. The drawer is `position:fixed` over a page it does not control, so anything it declines to handle, the page handles instead. A wheel it cannot use scrolls the page behind it, and `overscroll-behavior` cannot fix that half alone, because the property governs a scroller that has REACHED its edge and not one that never overflowed. Below 820px the drawer stops docking and covers the page outright, so "go back to the page" stops existing as an action and only "close the chat" remains. The VS Code panel docks at every width, and that one difference is what makes it read as part of the editor rather than as a thing on top of it. Read from the transcript 260801 and correcting how this was filed: below-820px is not an edge case to rule on later, it is the machine JL was holding when he hit the last five defects of the day (" ", 20:16). The phone is a usage mode, so docking at every width is a requirement rather than a fork. `build.py` embeds the chat into every self-contained board page, so the view is re-instantiated on each one and dies with it. That is the shared root of three separate-looking bugs: the drawer opened at the top because the replay ran while it was still `display:none`, a reload rebuilt it as the chat box while the terminal held the session, and navigating away aborted the turn. It is also why a fix does not reach the reader until the page is reloaded, which is the same coupling seen from the maintenance side. The VS Code split is the target and half of it is already ours: the extension host is a separate process that never touches the DOM, and M1's SessionHost is exactly that. The transcript is saved when a turn reaches `done`, and `done` is the last thing the stream ever sends, so it is precisely what a cut connection loses. A turn that ends any other way leaves the question saved and the reply gone, which is why an answer seemed to arrive only after the next message was sent. The same fragility appears in the small: the send path wrote the log under a bare id while the load path read a group under `G:<id>`, so a group's history was written to one key and looked for under another. Both carry one lesson, which is that a record with a single writing moment has a single moment in

which to be lost. Corrected 260801 from the transcript: this page dates the defect to 260801 and to having been found "by USING it", and it is a day older than that. On 260731 between 14:34 and 15:45 the same four probes were sent fourteen times, seven of them the identical "Narrate one short sentence before each step" and three the identical "Reply with exactly: HELLO", with no complaint attached to any of them. The re-sends WERE the symptom and nobody read them as one, which is the strongest argument on this page for the repeatable check: a defect that only shows up as a person quietly retrying is invisible to every method except driving it. The first three rows are state a reader can LOSE; this one is a control a reader never HAD, and it is the row that most directly answers why a terminal felt better. A terminal hands you scrollbar, a session list and a status line for free because the emulator has always had them, while every one of those is a thing the drawer must be handed one at a time. The evidence is that each arrived only when JL asked for it by name: the session picker on 260801 17:35 ("button Sessions"), the context meter at 17:46 ("context usage"), and both took an afternoon each. What is still missing is @-mentions and plan mode, and they stay last on purpose, because they are affordances on top of a harness whose state problems are the thing actually felt.

9 The one build that closes them, and it already exists next door

QD3 terminal - already right	QD2 chat - the gap (closed by R1)
PTY master fd	claude_agent_sdk
one reader thread term.py:195	one HTTP handler
ring bytearray 256KB	self.wfile.write() chat.py:637
RING_CAP base.py:65	the response socket ITSELF
+-> client A	no ring
+-> client B	no client list
+-> client C	no replay
reconnect ws_send(b"0"+ring) term.py:679	reader leaves emit() fails
grace deadline ,	stop.set() the turn DIES

The target was not a new architecture, it was the one QD3 shipped: a terminal survives a reload because its bytes go to a RING that clients attach to, while chat's bytes went straight down the socket of whoever happened to ask. That single asymmetry was the mechanism under three of the five rows above, and it is why the VS Code panel felt different rather than merely nicer. - R1 · the ring, ported from `term.py` — BUILT 260801, proven in a real browser 260802 `live/turnring.py` is the module: one `Turn` per question key, events carrying a monotonic `n`, a 1MB/20k cap that trims from the front and reports a `gap` rather than a silently short stream, and a 600s grace window. `emit()` now pushes into it and `chat()` spawns a runner thread, so the request is no longer the turn's owner but its FIRST READER; `POST /_board/attach {file, cursor}` is how the second one joins. Proven by `checks/ring_e2e.py`, which is shaped like the complaint rather than like the code: start a turn, hang up the socket the way navigating does, re-attach at the cursor, and demand the rest. It gets it — 117 events and an 11k-character answer that the original reader never saw. Two defects the wire test could not see and a real browser did, which is the rule working: a FINISHED ring was re-attached on every 25s heartbeat and repainted its answer each time (fixed: attach declines a finished turn, because once it ends the transcript is the right source), and the cursor was keyed on `logKey()`, which folds in a session id that CHANGES mid-turn (fixed). The last client defect, attaching at a cursor near zero so the drawer replayed instead of resuming, closed 260802: `checks/guichat.mjs` T6 reloads mid-turn and rejoins at cursor 137 against a pre-reload

cursor of 99 (States A9.1). `emit()` writes into a per-session outbox with a monotonic cursor and fans out to every attached client, instead of writing to one `wfile`. A departed reader stops being an event: no `stop.set()` at `chat.py:628`, no `fut.cancel() + host().evict()` in the `finally`, and `gate()` keeps answering permission requests for the life of the TURN rather than the life of the request. Closes: the turn dying on navigate, reload, config switch or timeout; the ten-minute turn; and . - R2 · the retained view — RETIRED 260801 to QD5, which had already built it The plan was to stop the drawer being rebuilt per page by making it a retained view attached to the ring by cursor, VS Code's webview answer to " ". Reading QD5's shell rather than its summary closed the row: the chat frame's src IS `<page>.html?pane=chat`, so the drawer is ALREADY its own document and no page navigation can reach it. That is the same outcome one layer up, and a better one, because it removes the rebuild instead of recovering from it. What survives here is only the half QD5 does not touch, which is that a replayed session still paints in a poorer format than a live one; that stays in §8's second row as its own item. - R3 · dock at every width — RETIRED 260801 to QD5, same reading `shell.py`'s own stylesheet already does it: `@media(max-width:820px)` switches the grid to `grid-template-rows:1fr 5px 1fr` and hides only the page list, so page and chat are both visible on a phone. The fork this page carried, overlay-with-a-way-back against collapse-to-a-tab, is answered by neither: the shell simply stacks them. So §9 is ONE build, not three, and the two that left were not descoped but already delivered next door. R1 stands alone because the turn dying with its HTTP response is a SERVER fact, three sites in `live/chat.py`, and no arrangement of frames reaches it. The blocker is named and is not technical: QD1's HOLD keys on the SCOPE path while the Law's stated reason is the shared jsonl, and a resumable channel redefines "one live window", so two named sessions on one page cannot be live together until that Law is re-ruled. CORRECTION, 260801, mine to make: this page briefly warned that R1's held `/_board/attach` would compete with QD5's `/_events` for the browser's six connections per origin. That was written from QD5's prose and is wrong. `/_events` holds nothing: it is one small JSON ask every 400 ms on a pooled connection, and `shell.py`'s own docstring explains that a held stream was built FIRST and removed precisely because it cost a connection. The budget question is still real but smaller and differently shaped: the held connections are the terminal's WebSocket and R1's own two, the turn stream and an attach.

10 Using it smoothly, and what proves each step

TWO DOORS TO THE SAME CHAT

```
.../QD2-chat-sdk.html?split    > three panes . strip: >_ TUI Chat . GUI Chat
.../QD2-chat-sdk.html         > the one-page board . bottom-right . pick GUI
```

THE TEN THINGS A READER DOES

	proven by
open it	the strip, or the button T1 . T1b . T13
ask	the answer renders as md T3
read back while it works	scrolling up is respected T5
stop it	, one honest line T12
leave mid-turn	it keeps running, you rejoin T6
move to another page	the chat stays on its own T10
change session	, or New session T11
change model or tier	Settings, remembered T16
use it on a phone	it fits, Page gets you back T14
hand it to the TUI	>_ TUI, and GUI takes back T17

Every row above is a gesture, not a feature, and each one is held by an assertion in `checks/guichat.mjs` so it cannot quietly stop working. Read this section as the manual and the test list at once: if a row here is wrong, the check that names it is the thing to run. ****Opening it.**** A board url with `?split` gives three panes, and the strip at the top carries `>_ TUI Chat` and `GUI Chat`. A plain board url gives the original one-page board, where the chat is the button at the bottom right and that button asks which of the two you want. Being in the split is remembered, so a plain url after using the split gives you the split back; `?plain` is how you ask for the single page on purpose. ****Asking, and reading while it works.**** The answer renders as markdown while it streams, and tool calls appear as their own cards. Scrolling up during a turn is respected: the next token will not drag you back down. Press to stop, and it says one thing about stopping rather than two. ****Leaving, and coming back.**** This is the part that used to lose work and no longer does. A turn does not belong to the browser window that started it: navigate away, reload, or lock your phone, and it keeps running on the server. When you come back the drawer rejoins the turn already in progress and the answer lands. Moving the page pane to another page does not touch the chat pane at all. ****Sessions.**** One question can hold many conversations. `Sessions` lists them, `New session` starts a fresh one, and picking an old one shows its real history read back from disk. Switching away and back returns the same messages in the same order. ****The two chats are one question.**** QD1's Law is one live window per question, so the GUI and the TUI hand the same session back and forth rather than running side by side. The strip's two buttons are how you do it, and the handover keeps the transcript.